

```
print("Hello, World!")
```



LINGUAGEM DE PROGRAMAÇÃO PYTHON



AULA 05

**SANTA
CRUZ**
CENTRO UNIVERSITÁRIO

ANTERIORMENTE

- **Condições lógicas;**
- **Onde utilizar estruturas condicionais;**
- **Estrutura condicional simples (if);**
- **Estrutura condicional composta (elif e else);**
- **match...case**
- **Apresentação dos tipos de laços de repetição;**
- **Estrutura de looping (While e For);**

PRÓXIMOS PASSOS

- **Break e Continue;**
- **Apresentação de funções;**
- **Exemplo de utilização;**
- **Estrutura de uma função em Python;**
- **Parâmetros;**
- **Escopo de variáveis.**
- **Recursividade;**

**Como ignorar uma iteração ou
parar a execução caso
aconteça algo?**

continue

- O comando **continue** é usado dentro de laços de repetição (**for** e **while**) para interromper a iteração atual e avançar para a próxima.
- Ao encontrar o **continue**, o Python ignora todas as linhas de código abaixo dele dentro do bloco do laço e volta para o início da estrutura de repetição para executar a próxima iteração.

Quando usar

- Para ignorar casos específicos ou tratar exceções sem parar o loop;
- Filtrar valores: Ignorar elementos específicos em uma coleção sem interromper o loop inteiro;
- Evitar aninhamento excessivo: Tratar casos de exceção ou ignorar dados inválidos logo no início da iteração;

continue

```
for numero in range (1,10):  
    if numero %2 == 0:  
        continue  
    print(f"Número Ímpar: {numero}")
```

Número Ímpar: 1

Número Ímpar: 3

Número Ímpar: 5

Número Ímpar: 7

Número Ímpar: 9

break

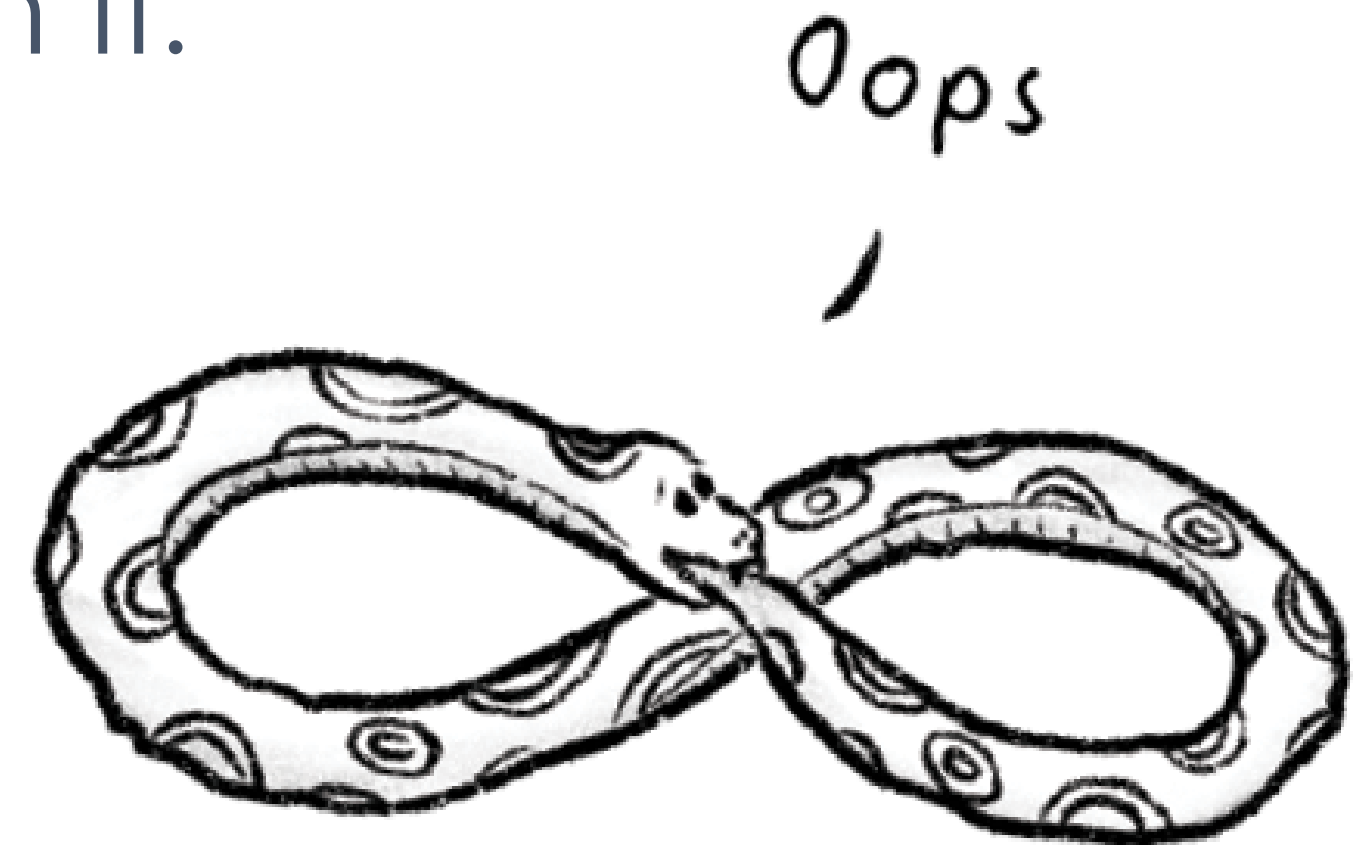
- É usado para interromper e encerrar imediatamente a execução do laço, independentemente de a condição principal ter sido concluída ou não;
- Ao encontrar o break, o Python sai na hora da estrutura de repetição e transfere a execução do programa para a primeira linha de código que estiver fora e logo após o laço.



Fonte: Putten e Putten (2026)

Quando usar

- Ideal para parar a execução assim que um item for encontrado;
- Evita rodar iterações desnecessárias.
- Saída antecipada: Permite criar condições de parada customizadas com if.



break

```
print("Procurando o número 5...")

for numero in range(1, 11):
    print(f"Testando o número: {numero}")

    if numero == 5:
        print("Achei o 5! Parando o loop.")
        break # O break destrói o loop. O 6, 7, 8... nunca serão testados.

print("Fim da busca.")
```

```
Procurando o número 5...
Testando o número: 1
Testando o número: 2
Testando o número: 3
Testando o número: 4
Testando o número: 5
Achei o 5! Parando o loop.
Fim da busca.
```

continue / break

- O break interrompe a estrutura de repetição inteira, já o continue interrompe apenas uma iteração da estrutura de repetição

Característica	continue	break
Ação no Loop	Pula para a próxima iteração	Encerra e sai do laço
Execução Restante	Cancela apenas o final da rodada atual	Cancela todas as próximas rodadas
Objetivo	Ignorar elementos ou casos específicos	Parar o processamento antecipadamente
Uso Comum	Filtrar dados indesejados	Parar a busca ao achar o resultado

Atividade Prática 01 Aula 05

Sistema de Monitoramento de Tráfego de Rede

Objetivo: Você está desenvolvendo um script para analisar pacotes de dados que transitam por um servidor da empresa. O sistema recebe uma lista com o tamanho dos pacotes (em Megabytes) e precisa processá-los individualmente seguindo algumas regras de segurança.

Dados de Entrada:

Utilize a seguinte lista de pacotes recebidos:

pacotes = [12, 0, 25, 40, -1, 18, 50]

- Escreva um programa usando um laço para percorrer a lista de pacotes e aplique as seguintes regras:
- Ignore pacotes com tamanho igual a 0, pois eles não contêm informação relevante.
- Se o sistema encontrar um pacote com valor menor que zero (ex: -1), exiba uma mensagem de alerta e interrompa o processamento (pois indica uma falha de segurança).
- Para os demais pacotes, exiba a mensagem: "Pacote de [X] MB processado com sucesso."

O que são funções?

- Uma função em Python é um bloco de código organizado e reutilizável responsável por realizar uma tarefa específica;
- Em vez de repetir as mesmas linhas de código ao longo do seu programa, você encapsula essa lógica dentro de uma função e a chama sempre que necessário;
- São usadas para organizar o código e evitar repetições desnecessárias;

Por que usar funções?

- Reutilização de Código: Escreva a lógica uma vez e use em múltiplos lugares.
- Modularização: divide um programa grande em partes menores e organizadas.
- Legibilidade: torna o código mais fácil de entender e manter.
- Facilidade de manutenção: se precisar alterar a lógica, basta mudar apenas a função.



Fonte: Putten e Putten (2026)

Exemplo de uso

- **Cenário:** Imagine um *e-commerce* onde você precisa aplicar um cupom de desconto de 10% no carrinho de compras.
- **Sem Função:** Toda vez que o usuário adiciona um item (uma camiseta, um tênis, um livro), você precisaria reescrever o cálculo do desconto e do frete no código. Se a regra do cupom mudar para 15%, você terá que alterar o código em vários lugares diferentes.
- **Com Função (Ideal):** Você cria a função `calcular_preco_final(preco, desconto)` uma única vez. Sempre que um produto for comprado, basta chamar essa função. Se a regra mudar, você edita apenas uma linha de código.

Como utilizar funções em Python?



Fonte: Putten e Putten (2026)

Sintaxe Básica

- Utiliza-se a palavra-chave **def**, seguida pelo **nome da função** e parênteses **()**: .
- Para chamar uma função (executar o código) utilizamos o **nome** seguido de **parênteses**.
- Uma mesma função pode ser chamada múltiplas vezes.
- **def**: Indica o início da definição da função.
- **Parâmetros/Argumentos**: Valores passados para dentro da função para que ela possa processá-los.
- **Corpo da Função**: O bloco de código indentado que executa a instrução.

Sintaxe Básica

```
def nome_da_funcao(parametro):  
    # Bloco de código
```

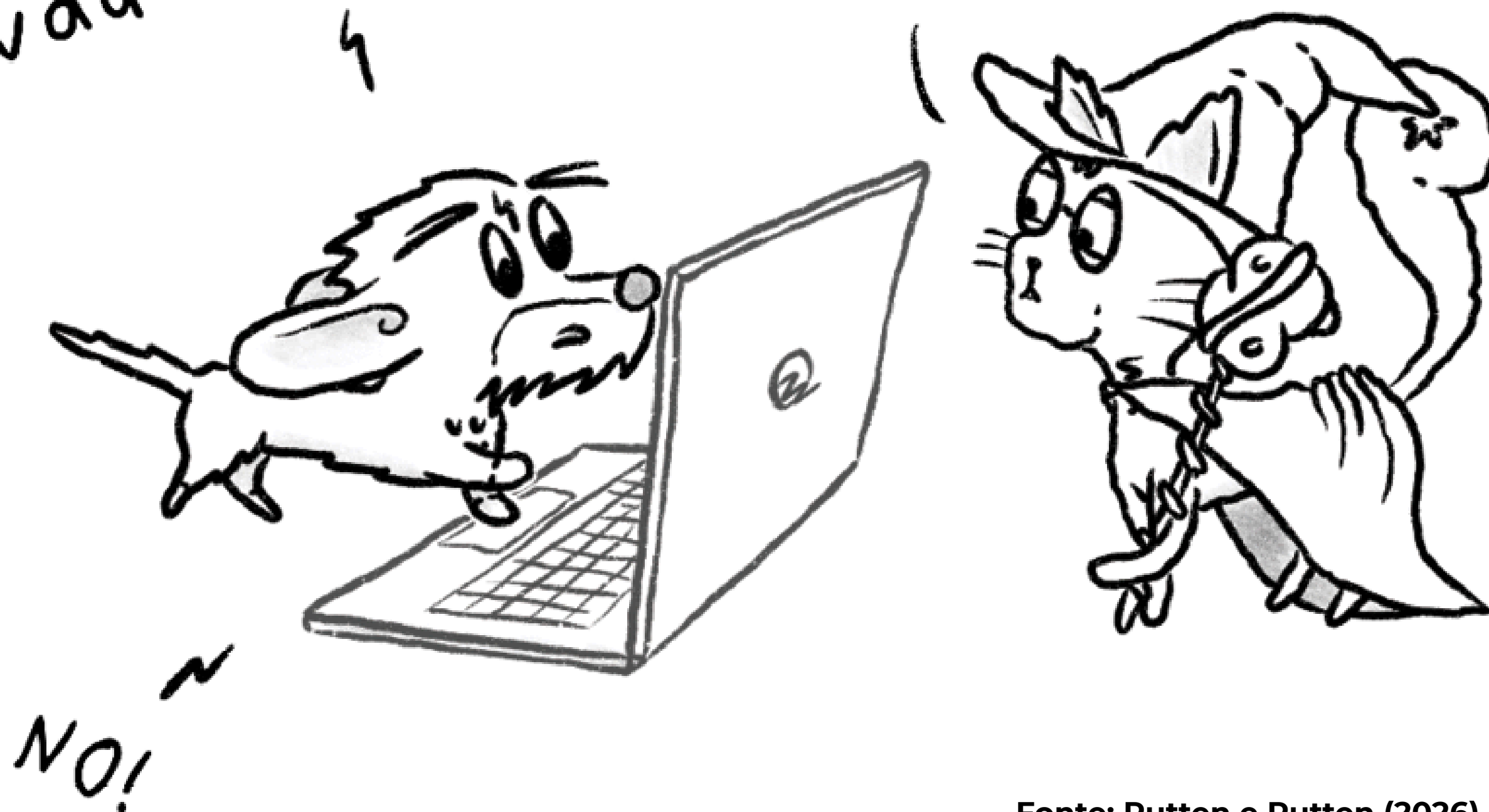
```
def forjar_arma(arma, dono):  
    print(f" A arma {arma} foi forjada com sucesso para {dono}!")
```

```
forjar_arma("Ferroada", "Frodo")  
forjar_arma("Andúril", "Aragorn")
```

A arma Ferroada foi forjada com sucesso para Frodo!
A arma Andúril foi forjada com sucesso para Aragorn!

avada_kedavra()

You mean
abracadabra()..?



Parâmetros e argumentos de funções

- Parâmetros: São as variáveis criadas na definição da função (os "espaços reservados" para os dados). A variável declarada entre parênteses no **def** para receber um dado.
- Argumentos: São os valores reais passados dentro dos parênteses na chamada da função (os dados concretos enviadas para execução).
- Em resumo: O **parâmetro** é o conceito/estrutura; o **argumento** é o dado real.

Parameters are like empty containers, and arguments are what you put into them.



Fonte: Putten e Putten (2026)

Parâmetros e argumentos de funções

'nome' é o PARÂMETRO

```
def saudar(nome):  
    print(f"Olá, {nome}")
```

'Aragorn' é o ARGUMENTO

```
saudar("Aragorn")
```

Olá, Aragorn

Parâmetros com valor padrão

- Valor Padrão (Default): Permite definir um valor padrão caso nenhum argumento seja passado na chamada da função.
- Sintaxe do Sinal = : O valor padrão é atribuído diretamente na criação do parâmetro (**parametro = valor**).
- Regra de Ordem: Parâmetros com valor padrão devem ficar sempre ao final (à direita) da lista de parâmetros (primeiro obrigatórios, depois opcionais).
- Flexibilidade: Permite chamar a mesma função com ou sem determinados argumentos.

Parâmetros com valor padrão

```
# 'titulo' tem o valor padrão "Guerreiro"
def criar_personagem(nome, titulo="Guerreiro"):
    print(f"{nome}, o {titulo}")

criar_personagem("Legolas", "Elfo")    # Saída: Legolas, o Elfo
criar_personagem("Gimli")              # Saída: Gimli, o Guerreiro
```

```
Legolas, o Elfo
Gimli, o Guerreiro
```

Argumentos nomeados

- Identificação por nome: os argumentos são passados informando diretamente o **nome** do parâmetro (**parametro = valor**).
- Independência de ordem: não é necessário seguir a ordem exata em que os parâmetros foram declarados no **def**.
- Clareza no código: deixa a chamada da função mais explícita e fácil de entender.
- Leitura facilitada: evita confusão sobre o que cada valor representa ao chamar funções com muitos parâmetros.

Argumentos nomeados

```
def criar_personagem(nome, classe, reino):  
    print(f"{nome} é {classe} de {reino}")
```

```
# Chamada usando argumentos nomeados (fora da ordem original):  
criar_personagem("Gondor", "Aragorn", "Guerreiro")  
criar_personagem(reino="Gondor", nome="Aragorn", classe="Guerreiro")
```

Gondor é Aragorn de Guerreiro
Aragorn é Guerreiro de Gondor

Recebendo múltiplos argumentos

- Caso você não saiba quantos argumentos irá receber, é possível receber uma quantidade indeterminada.
- Sintaxe com Asterisco (*): Definido usando ***args** na criação da função.
- Uso do ***args**: O nome args é uma convenção (o importante é o asterisco *).
- Os argumentos passados para ***args** são armazenados como uma **tupla** dentro da função
- Iteração simples: Permite percorrer todos os valores recebidos usando um laço **for**

Recebendo múltiplos argumentos

```
# 0 *args recebe quantos nomes forem enviados
def convocar_comitiva(*membros):
    print("Membros convocados:")
    for membro in membros:
        print(f"- {membro}")

# Chamando com 3 argumentos
convocar_comitiva("Frodo", "Sam", "Gimli")
```

Membros convocados:

- Frodo
- Sam
- Gimli

Recebendo múltiplos argumentos nomeados

- Podemos ainda, receber uma quantidade indeterminada de argumentos nomeados;
- Sintaxe com dois asteriscos (**): utilizando ****kwargs** na criação da função.
- **Estrutura de Chave e Valor:** Os dados ficam guardados como pares compostos por um nome (**chave**) e um conteúdo (**valor**).
- **Acesso por Chaves:** Permite acessar as chaves e os valores passados de forma dinâmica.
- Permite enviar quantas combinações de nome = valor forem necessárias.
- **Organização dos Dados:** Associa cada informação a uma etiqueta identificadora.

Recebendo múltiplos argumentos nomeados

```
# O **kwargs recebe características dinâmicas
def ficha_personagem(**atributos):
    for chave, valor in atributos.items():
        print(f"{chave.capitalize()}: {valor}")

# Chamando a função com diferentes argumentos nomeados
ficha_personagem(nome="Aragorn", classe="Ranger", reino="Gondor")
```

Nome: Aragorn
Classe: Ranger
Reino: Gondor

Funções com retorno

- Quando uma função não tem retorno, ela executa o bloco de código e retorna **None** automaticamente. Esse tipo de função é ideal para realizar ações diretas, como exibir uma mensagem na tela, salvar um arquivo ou atualizar algo no sistema.
- Porém, as funções podem retornar um resultado.
- Útil quando queremos usar o valor em outras partes do código.
- Para retornar um valor é utilizado a palavra chave **return**.
- O valor retornado pode ser de qualquer tipo (int, string, lista, função, etc).

Funções com retorno

```
def contar_flechas(alvos_atingidos):  
    total = alvos_atingidos * 2  
    return total
```

```
print(f'Legolas usou {contar_flechas(5)} flechas )
```

Legolas usou 10 flechas

I'm going to
need coffee for
this.



Atividade Prática 02 Aula 05

O Sistema da Comitiva do Anel

Objetivo: Criar um script para gerenciar os membros da Comitiva e preparar seus equipamentos utilizando funções.

Etapa 1:

Registro do Herói (Parâmetros com valor padrão e return):

O Crie uma função que receba o nome e a classe do personagem e, opcionalmente, o seu reino.

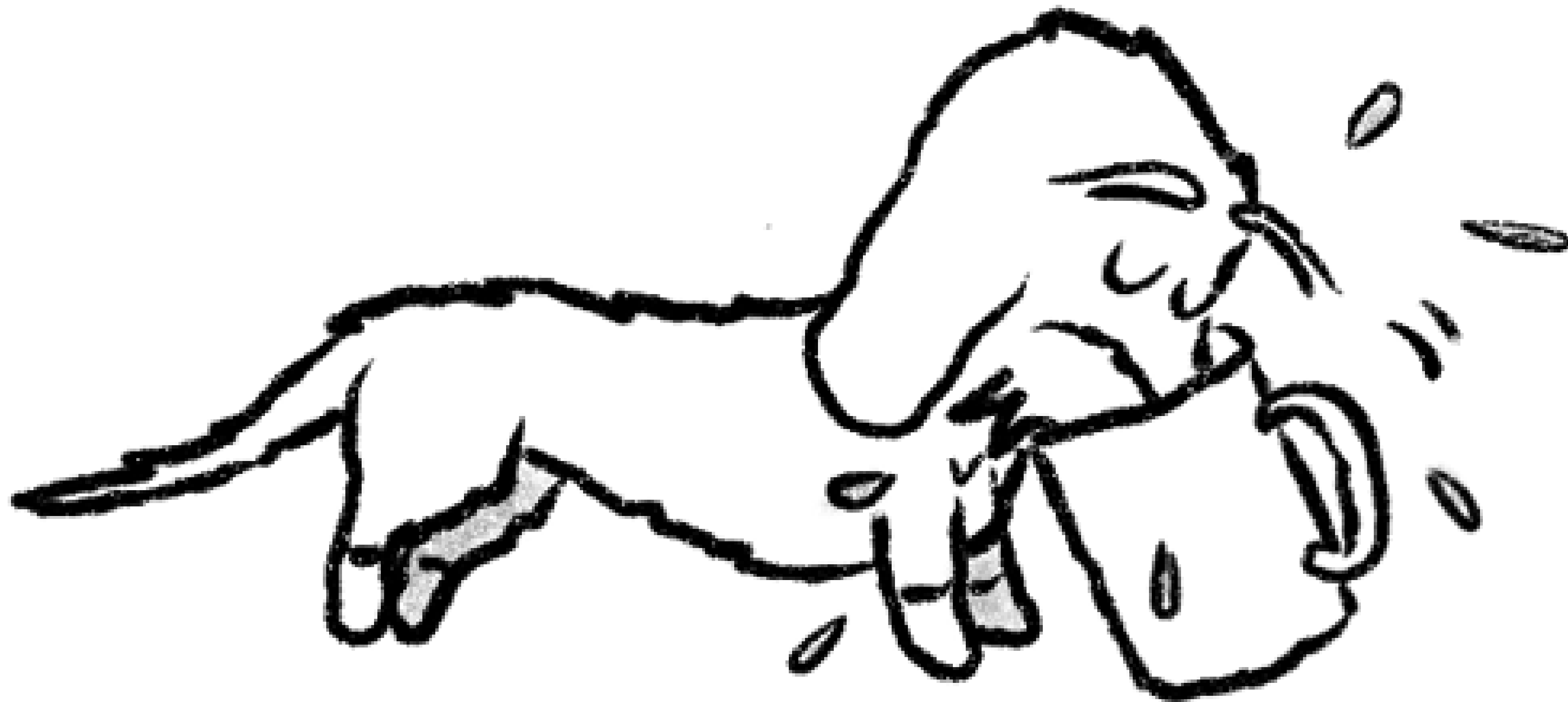
- A função deve retornar um texto formatado
- Teste: invoque a função com e sem o parâmetro reino.

Etapa 2:

A Chamada das Armas:

Crie uma função que receba o nome do herói e uma quantidade indeterminada de itens.

- A função deve imprimir o nome do herói e listar cada item recebido com um marcador (-).
- Teste: chame a função passando 3 itens.



Atividade Prática 02 Aula 05

Etapa 3:

Ficha Detalhada do Personagem:

Crie uma função para registrar estatísticas de combate dinâmicas.

- A função deve exibir o nome do personagem e percorrer todos os atributos passados (ex: força=80, agilidade=95) imprimindo o nome do atributo e o seu valor.
- Teste: chame a função usando argumentos nomeados.

Desafio Bônus (Controle de Fluxo):

A Jornada:

Crie um laço que percorra uma lista de perigos no caminho para Mordor:

- Se o perigo for "Nevoeiro": ignorar o combate e seguir viagem.
- Se o perigo for "Balrog": interromper a jornada imediatamente (Gandalf avisa para fugir!).



Escopo de variáveis



Fonte: Putten e Putten (2026)

Variáveis locais

- **Escopo Restrito:** Criadas dentro de uma função e acessíveis apenas dentro do seu bloco de código. Tentativas de acessar a variável fora da função resultam em erro de sistema (**NameError**).
- **Vida Útil Temporária:** Nascem quando a função é executada e são destruídas assim que a função termina.
- **Isolamento de Dados:** Variáveis locais com o mesmo nome em funções diferentes não interferem uma na outra. Evita alterações acidentais em valores usados em outras partes do código.

Variáveis locais

```
def preparar_viagem():  
    suprimentos = "Lembas" # Variável Local  
    print(f"Carregando: {suprimentos}")
```

```
preparar_viagem()  
print(suprimentos)
```

```
# Tentar usar 'suprimentos' fora da função vai gerar erro:  
# print(suprimentos) -> NameError!
```

Carregando: Lembas

```
NameError                                Traceback (most recent call last)  
/tmp/ipykernel_1277/2819504893.py in <cell line: 0>()  
NameError: name 'suprimentos' is not defined
```

Variáveis globais

- **Escopo Geral:** Declaradas fora de qualquer função e acessíveis em todo o programa.
- **Vida Útil Permanente:** Nascem quando são criadas e permanecem na memória até a execução do código terminar.
- **Leitura Livre:** Qualquer função dentro do código consegue ler e usar o valor de uma variável global.
- **Atenção às Modificações:** Tentar alterar uma variável global dentro de uma função requer o uso da palavra-chave **global**.

Variáveis globais

```
reino_principal = "Gondor" # Variável Global
```

```
def mostrar_reino():  
    # Acessa a variável global sem problemas  
    print(f"Bem-vindo ao reino de {reino_principal}!")
```

```
mostrar_reino()  
print(reino_principal) # Também acessível no programa principal
```

```
Bem-vindo ao reino de Gondor!  
Gondor
```

Modificando variáveis globais e aninhadas

- Exige o uso da palavra-chave **global** para permitir alterações dentro de uma função.
- Caso desejemos modificar uma variável que não é global, mas foi definida em outra função mais externa usamos **nonlocal**.
- **A palavra-chave nonlocal permite alterar variáveis de uma função externa (pai), sem torná-las globais.**
- **Comportamento Padrão:** Sem global ou nonlocal, qualquer atribuição de valor cria apenas uma nova variável local.

Modificando variáveis globais

```
ouro_do_reino = 100  # Variável Global
```

```
def gastar_ouro(quantidade):  
    global ouro_do_reino  # Permite alterar a variável global  
    ouro_do_reino -= quantidade
```

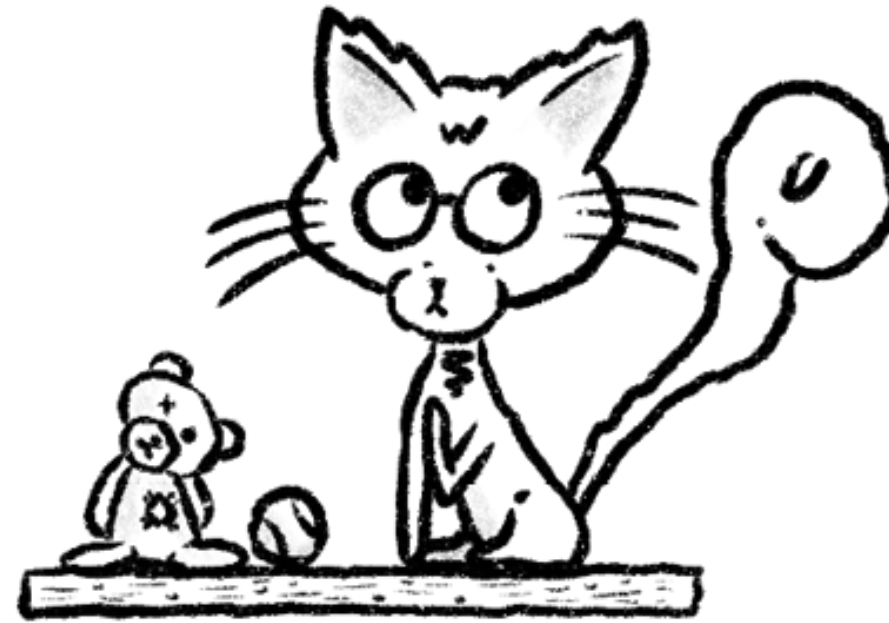
```
gastar_ouro(30)  
print(f'Ouro restante: {ouro_do_reino}")  # Saída: Ouro restante: 70
```

Ouro restante: 70

Modificando variáveis aninhadas

```
def comitiva():  
    membros = 9  # Variável da função externa  
  
    def membro_caiu():  
        nonlocal membros  # Modifica a variável da função pai (comitiva)  
        membros -= 1  
  
    membro_caiu()  
    print(f"Membros restantes: {membros}")  # Saída: 8  
  
comitiva()
```

Membros restantes: 8



Ball?



I'm sorry,
Wiesje.

It's out of
your scope.

Documentação de Funções

Documentação de funções

- A documentação de uma função é um texto explicativo que descreve o que a função faz, quais são seus parâmetros e o que ela retorna.
- **Facilidade de Leitura:** Ajuda outros desenvolvedores (e você no futuro!) a entenderem o código sem precisar ler toda a lógica interna.
- **Uso de Aspas Triplas:** Em Python, a documentação é escrita usando aspas triplas (""" ... """) logo na primeira linha abaixo do **def**.
- **Consulta Integrada:** Permite usar a função **help(nome_da_funcao)** ou passar o mouse sobre ela no editor para ler o manual de uso.

Docstrings

```
def calcular_dano(forca, multiplicador=1.5):  
    """  
    Calcula o dano final do ataque do herói.  
  
    Parâmetros:  
    forca (int): Valor base de força do herói  
    multiplicador (float): Bônus da arma (padrão 1.5)  
  
    Retorna:  
    float: Dano total do ataque  
    """  
    return forca * multiplicador
```

```
# Consultando a documentação da função no terminal/IDE  
help(calcular_dano)
```

Funções Lambda λ



Fonte: Putten e Putten (2026)

O que são?

- **Compacta:** Forma rápida de criar funções simples de uma única linha sem usar a palavra-chave **def**.
- Pode ter vários argumentos, mas apenas uma expressão.
- **Retorno Implícito:** O resultado da expressão é retornado automaticamente (não se usa a palavra **return**).
- **Sem Nome (Anônimas):** Não exigem um nome formal para existirem, sendo ideais para usos temporários.

Sintaxe

Definida pela palavra-chave **lambda**, seguida dos parâmetros, dois pontos e a expressão (**lambda** **parametros: expressao**).

```
# Função tradicional (com def)
```

```
def dobrar_dano(dano):  
    return dano * 2
```

```
# Mesma lógica usando lambda
```

```
dobrar_dano_lambda = lambda dano: dano * 2
```

```
# Testando
```

```
print(dobrar_dano(50))
```

```
print(dobrar_dano_lambda(50)) |
```

```
100
```

```
100
```

Atividade Prática 03 Aula 05

Monitoramento de TI

Objetivo: Criar um script para registrar alertas, aplicar configurações dinâmicas e processar chamados da infraestrutura.

Etapa 1: Registro de Servidor (global e Docstring)

- Crie uma variável global e uma função que use a palavra-chave *global* para controlar o total de servidores online.
- Adicione uma Docstring explicando a função, parâmetros e o impacto na variável global.
- Desative/ative 5 servidores e imprima o saldo de servidores ativos.

Etapa 2: Configuração de Conexão ao Banco de Dados Aninhada

- Crie uma função para gerenciar conexões e dentro dela, criar uma variável local.
- Crie uma função interna que use *nonlocal* para alterar a quantidade de conexões.
- Execução: Chame a função interna 2 vezes e exiba o limite de conexões restantes.

Atividade Prática 03 Aula 05

Etapa 3: Ajuste Temporário de Cota (Função lambda)

- Crie uma função com lambda para alterar a cota de armazenamento de arquivos em nuvem no servidor.
- Teste: chame a função alterando a cota.

Próximos Passos



**Listas, Dicionários, Conjuntos e Tuplas;
Tratamento de erros;
Bibliotecas no Python;**

**exit() ou
dúvidas?**

**SANTA
CRUZ**
CENTRO UNIVERSITÁRIO

OBRIGADA!

“Readability counts.”

The Zen of Python, by Tim Peters



Referências: PUTTEN, Maaïke van; PUTTEN, Imke van. Python Illustrated: Not another boring Python book, learn programming the fun way. Birmingham: Packt Publishing, 2026. [1, 2, 3]

SANTA CRUZ

CENTRO UNIVERSITÁRIO

Para mais
informações
acesso
o **QR CODE**
e saiba mais



 (41)3052-4900

 @unisantacruz

 /UniSantaCruzCtba

 unisantacruztem.com.br